

L2 - 13.3. Windows Internals

Task 1 - Introduction

Operating systems have a lot more technology and architecture behind them than we may see at first. With Windows machines making up a majority of corporate infrastructure, red teams need to understand Windows internals and how they can be (ab)used. The red team can (ab)use Windows to aid in evasion and exploitation when crafting offensive tools or exploits.

Task 2 - Processes

A process represents the execution of a program (an application can have one or multiple processes) and consists of at least one thread along with key components such as executable code, a unique ID (PID), a virtual address space, security contexts, environment variables, system object handles, and priority settings.

Process Component	Purpose
Private Virtual Address Space	Virtual memory addresses that the process is allocated.
Executable Program	Defines code and data stored in the virtual address space.
Open Handles	Defines handles to system resources accessible to the process.
Security Context	The access token defines the user, security groups, privileges, and other security information.
Process ID	Unique numerical identifier of the process.
Threads	Section of a process scheduled for execution.

Component	Purpose
Code	Code to be executed by the process.
Global Variables	Stored variables.
Process Heap	Defines the heap where data is stored.

Process Resources	Defines further resources of the process.
Environment Block	Data structure to define process information.

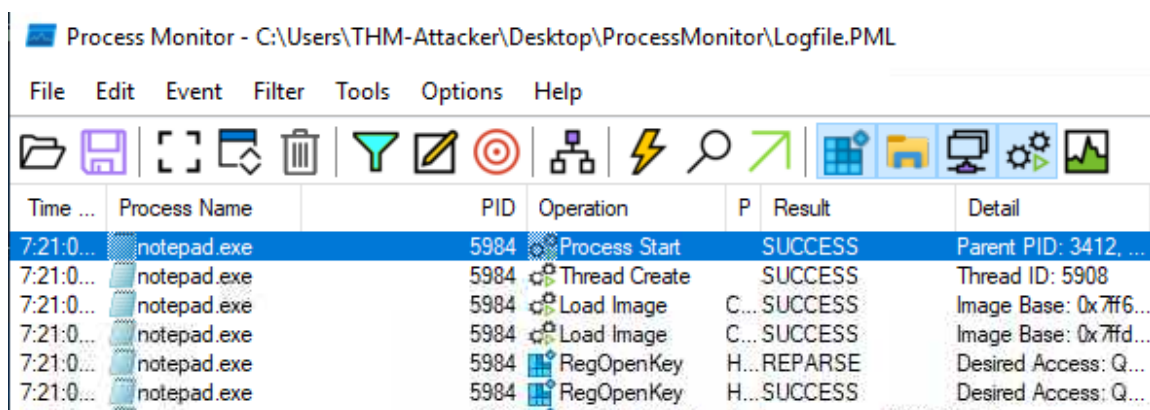
The task manager can report on many components and information about a process. Below is a table with a brief list of essential process details.

Value/Component	Purpose	Example
Name	Define the name of the process, typically inherited from the application	conhost.exe
PID	Unique numerical value to identify the process	7408
Status	Determines how the process is running (running, suspended, etc.)	Running
User name	User that initiated the process. Can denote privilege of the process	SYSTEM

Processes are at the core of most internal Windows components. **Procmon** (short for **Process Monitor**) is an advanced monitoring tool for Windows developed by Microsoft Sysinternals. It shows real-time file system, Registry, and process/thread activity. It essentially gives you a live look at what programs are doing under the hood on a Windows machine.

Answer the questions below

Open the provided file: "Logfile.PML" in Procmon and answer the questions below.
Run the procmon on desktop, after started procmon , open file logfile



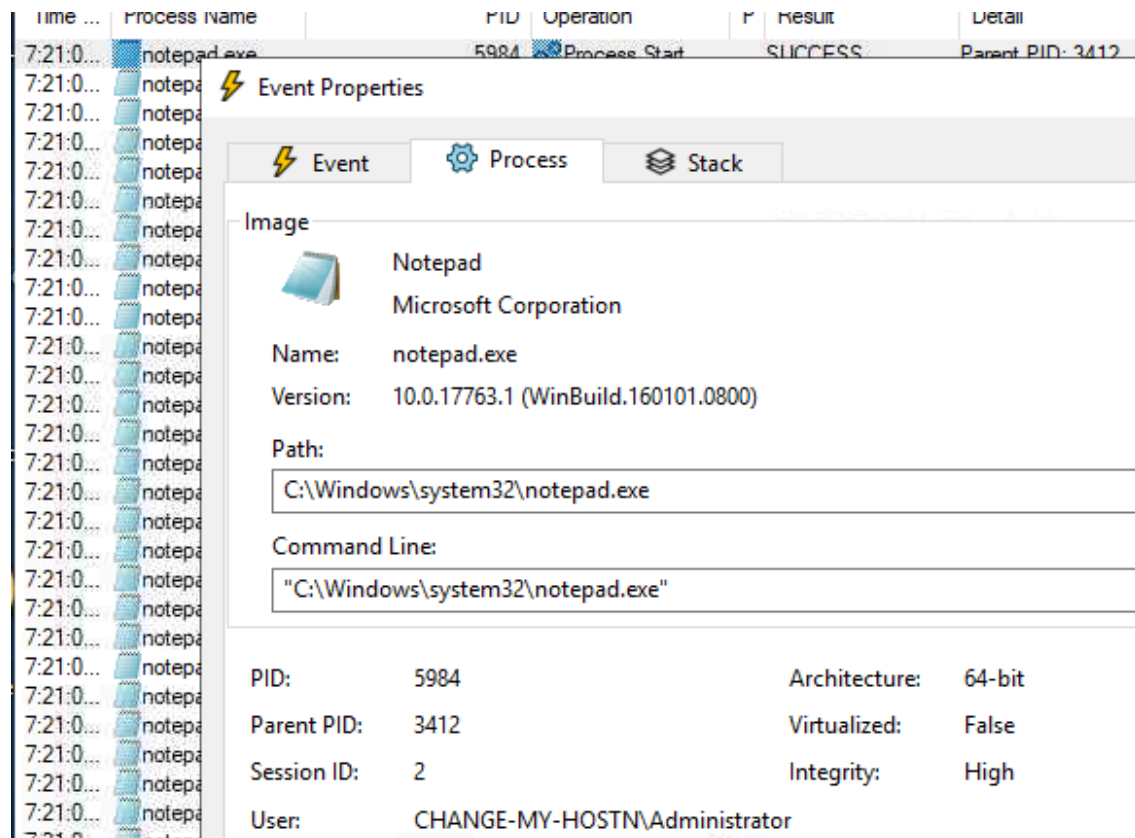
Time ...	Process Name	PID	Operation	P	Result	Detail
7:21:0...	notepad.exe	5984	Process Start		SUCCESS	Parent PID: 3412, ...
7:21:0...	notepad.exe	5984	Thread Create		SUCCESS	Thread ID: 5908
7:21:0...	notepad.exe	5984	Load Image	C...	SUCCESS	Image Base: 0x7ff6...
7:21:0...	notepad.exe	5984	Load Image	C...	SUCCESS	Image Base: 0x7ffd...
7:21:0...	notepad.exe	5984	RegOpenKey	H...	REPARSE	Desired Access: Q...
7:21:0...	notepad.exe	5984	RegOpenKey	H...	SUCCESS	Desired Access: Q...

What is the process ID of "notepad.exe"? 5984

What is the parent process ID of the previous process? 3412

What is the integrity level of the process?

High (Double-click a process, then click 'Process'.)



Task 3 - Threads

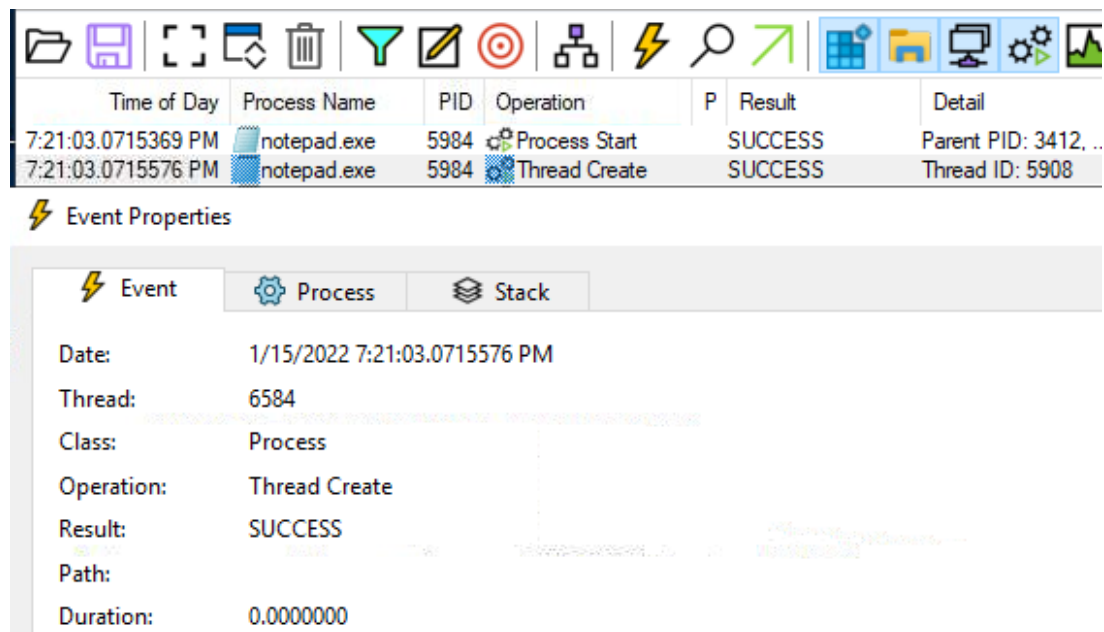
A **thread** is an executable unit that controls a process's execution and is scheduled by CPU and memory factors. Threads share their parent process's resources (like code and global variables) while keeping some unique data, making them a frequent target for attackers attempting malicious code execution.

Component	Purpose
Stack	All data relevant and specific to the thread (exceptions, procedure calls, etc.)
Thread Local Storage	Pointers for allocating storage to a unique data environment

Stack Argument	Unique value assigned to each thread
Context Structure	Holds machine register values maintained by the kernel

Answer the questions below

Open the provided file: "Logfile.PML" in Procmon and answer the questions below. What is the thread ID of the first thread created by notepad.exe? 5908



What is the stack argument of the previous thread? 6584

Task 4 - Virtual Memory

Virtual memory gives each process its own private address space, which a memory manager translates into physical memory. This prevents applications from interfering with or damaging each other. To handle high memory demands, the memory manager transfers (pages) virtual memory to the disk when an application exceeds its allocated physical memory.

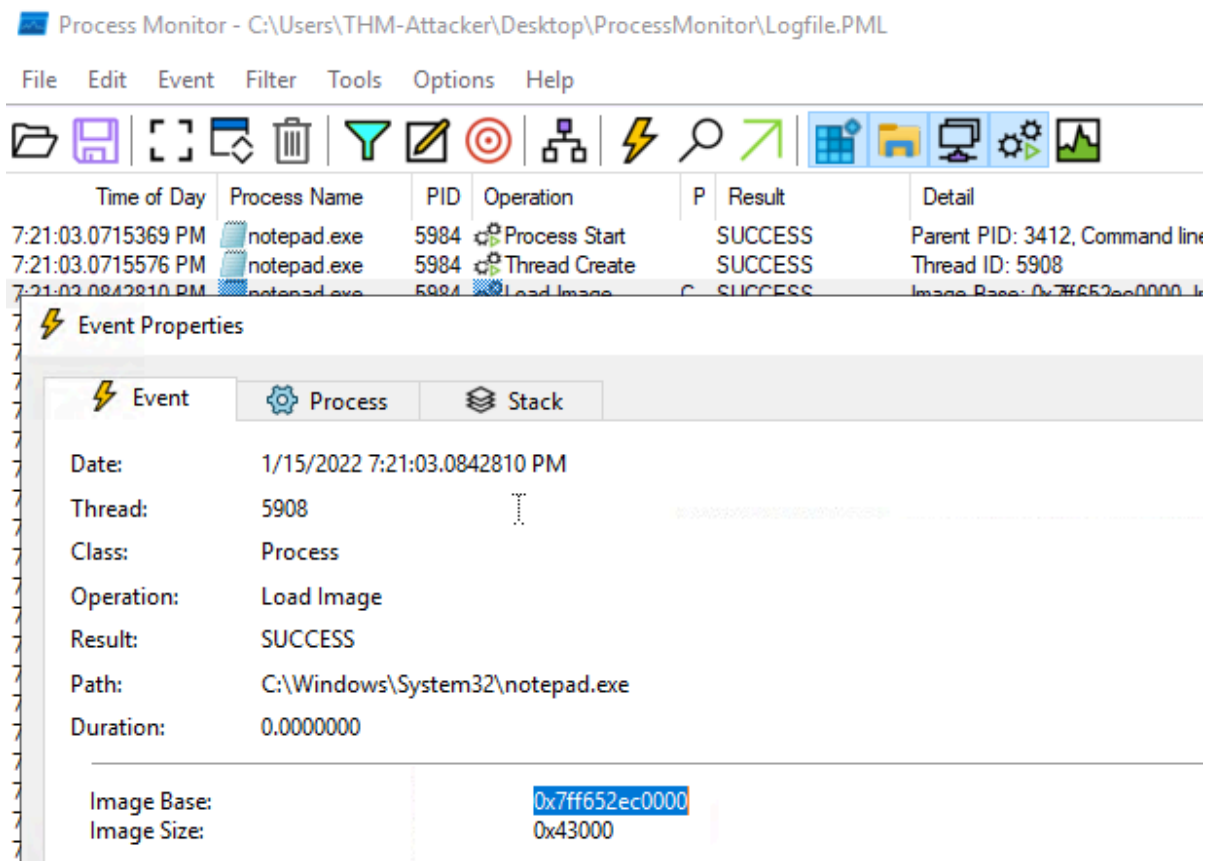
Address Space Layout: 32-bit systems: Max address space is 4 GB, split equally between the process (user space) and the operating system (kernel space). This can be adjusted using settings like `increaseUserVA` or AWE.

Answer the questions below

Read the above and answer the questions below. What is the total theoretical maximum virtual address space of a 32-bit x86 system? 4 GB

What default setting flag can be used to reallocate user process address space? increaseuserva

Open the provided file: "Logfile.PML" in Procmon and answer the questions below.



What is the base address of "notepad.exe"? 0x7ff652ec0000

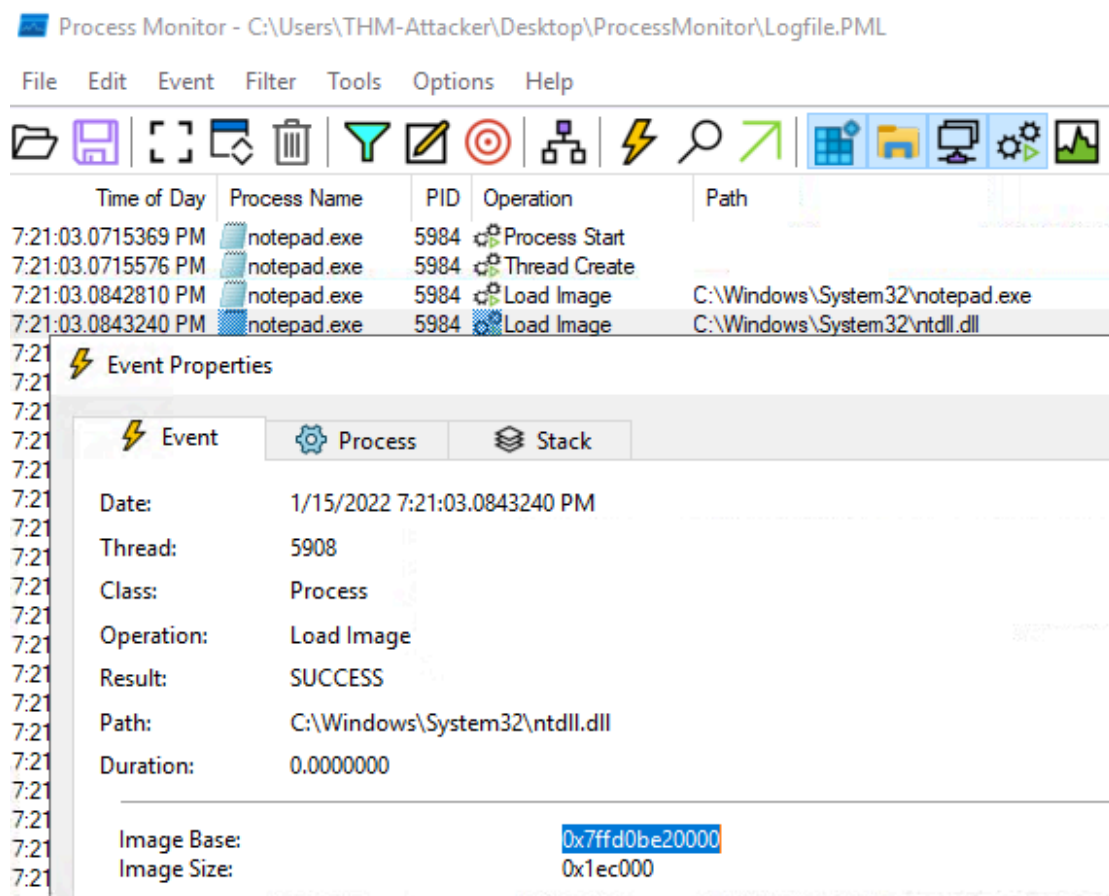
Task 5 - Dynamic Link Libraries

A Dynamic-Link Library (DLL) is a reusable library containing code and data that multiple programs can share simultaneously. Using DLLs improves application speed, modularity, memory efficiency, and disk space usage. Calls DLL functions explicitly using header (`.h`) and import library (`.lib`) files. Dynamically loads the

DLL while the application runs via APIs like `LoadLibrary` / `LoadLibraryEx` and retrieves function addresses using `GetProcAddress`.

Answer the questions below

Open the provided file: "Logfile.PML" in Procmon and answer the questions below. What is the base address of "ntdll.dll" loaded from "notepad.exe"?
0x7ffd0be20000



What is the size of "ntdll.dll" loaded from "notepad.exe"? 0x1ec000

Process Monitor - C:\Users\THM-Attacker\Desktop\ProcessMonitor\Logfile.PML

File Edit Event Filter Tools Options Help

Time of Day Process Name PID Operation Path Result Detail

7:21:03.0715369 PM	notepad.exe	5984	Process Start		SUCCESS	Parent PID
7:21:03.0715576 PM	notepad.exe	5984	Thread Create		SUCCESS	Thread ID:
7:21:03.0842810 PM	notepad.exe	5984	Load Image	C:\Windows\System32\notepad.exe	SUCCESS	Image Base

Event Properties

Event Process Stack

Image

Notepad
Microsoft Corporation

Name: notepad.exe
Version: 10.0.17763.1 (WinBuild.160101.0800)

Path:
C:\Windows\system32\notepad.exe

Command Line:
"C:\Windows\system32\notepad.exe"

PID: 5984 Architecture: 64-bit
Parent PID: 3412 Virtualized: False
Session ID: 2 Integrity: High
User: CHANGE-MY-HOSTN\Administrator
Auth ID: 00000000:000951cd
Started: 1/15/2022 7:21:03 PM Ended: (Running)

Modules:

Module	Address	Size	Path	Company	Version
notepad.exe	0x7ff652ec0000	0x43000	C:\Windows\System32\notepad.e...	Microsoft Cor...	10.0.17763.1 (...)
ntdll.dll	0x7ffd0be20000	0x1ec000	C:\Windows\System32\ntdll.dll	Microsoft Cor...	10.0.17763.172...

How many DLLs were loaded by "notepad.exe"? 51

Task 6 - Portable Executable Format

The Portable Executable (PE) format defines the structure, information, and data storage for Windows executables and object files (comprising PE and COFF files). PE data layout is most clearly visible by inspecting an executable's hex dump.

- **DOS Header:** Identifies the file type using the **MZ** signature to mark it as an **.exe**.
- **DOS Stub:** A default compatibility message (such as *"This program cannot be run in DOS mode"*) displayed if run in DOS, which doesn't affect standard file

functionality.

- **PE File Header:** Contains essential binary header information, including the file format, signatures, and image file headers (typically the least human-readable section).

```
000000E0 00 00 00 00 00 00 00 00 50 45 00 00 64 86 06 00 .....PE..d+..
000000F0 10 C4 40 03 00 00 00 00 00 00 00 00 F0 00 22 00 .Ä@.....ð."
00000100 0B 02 0E 14 00 0C 00 00 00 62 00 00 00 00 00 00 .....b.....
00000110 70 18 00 00 00 10 00 00 00 00 00 40 01 00 00 00 p.....@....
00000120 00 10 00 00 00 02 00 00 0A 00 00 00 0A 00 00 00 .....
00000130 0A 00 00 00 00 00 00 00 00 B0 00 00 00 04 00 00 .....°.....
00000140 63 41 01 00 02 00 60 C1 00 00 08 00 00 00 00 00 cA....`Á.....
00000150 00 20 00 00 00 00 00 00 00 00 10 00 00 00 00 00 . ....
00000160 00 10 00 00 00 00 00 00 00 00 00 00 10 00 00 00 .....
00000170 00 00 00 00 00 00 00 00 94 27 00 00 A0 00 00 00 ....."'...
00000180 00 50 00 00 10 47 00 00 00 40 00 00 F0 00 00 00 .P...G...@..ð...
00000190 00 00 00 00 00 00 00 00 A0 00 00 2C 00 00 00 .....
000001A0 20 23 00 00 54 00 00 00 00 00 00 00 00 00 00 00 #..T.....
000001B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001C0 10 20 00 00 18 01 00 00 00 00 00 00 00 00 00 00 . ....
000001D0 28 21 00 00 40 01 00 00 00 00 00 00 00 00 00 00 (!..@.....
000001E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

- **Image Optional Header:** Despite its name, this is a crucial component of the PE File Header that contains the **Data Dictionaries**, which point to the image data directory structure.
- **Section Table:** Defines the available sections and metadata within the file structure.
- **Sections:** Store the executable's actual operational contents, including executable code, imported functions, and data, all of which can be identified in a hex dump.

```
000001F0 2E 74 65 78 74 00 00 00 D0 0B 00 00 00 10 00 00 .text...Ð.....
00000200 00 0C 00 00 00 04 00 00 00 00 00 00 00 00 00 00 .....
00000210 00 00 00 00 20 00 00 60 2E 72 64 61 74 61 00 00 .... ..`.rdata..
00000220 76 0C 00 00 00 20 00 00 00 0E 00 00 00 10 00 00 v....
00000230 00 00 00 00 00 00 00 00 00 00 00 00 40 00 00 40 .....@..@
00000240 2E 64 61 74 61 00 00 00 B8 06 00 00 00 30 00 00 .data..._...0..
00000250 00 02 00 00 00 1E 00 00 00 00 00 00 00 00 00 00 .....
```

```

00000260  00 00 00 00 40 00 00 C0 2E 70 64 61 74 61 00 00  ....@..À.pdata..
00000270  F0 00 00 00 00 40 00 00 00 02 00 00 00 20 00 00  Ÿ....@..... ..
00000280  00 00 00 00 00 00 00 00 00 00 00 00 40 00 00 40  .....@..@
00000290  2E 72 73 72 63 00 00 00 10 47 00 00 00 50 00 00  .rsrc....G...P..
000002A0  00 48 00 00 00 22 00 00 00 00 00 00 00 00 00 00  .H...".....
000002B0  00 00 00 00 40 00 00 40 2E 72 65 6C 6F 63 00 00  ....@..@.reloc..
000002C0  2C 00 00 00 00 A0 00 00 00 02 00 00 00 6A 00 00  ,.... .....j..
000002D0  00 00 00 00 00 00 00 00 00 00 00 00 40 00 00 42  .....@..B

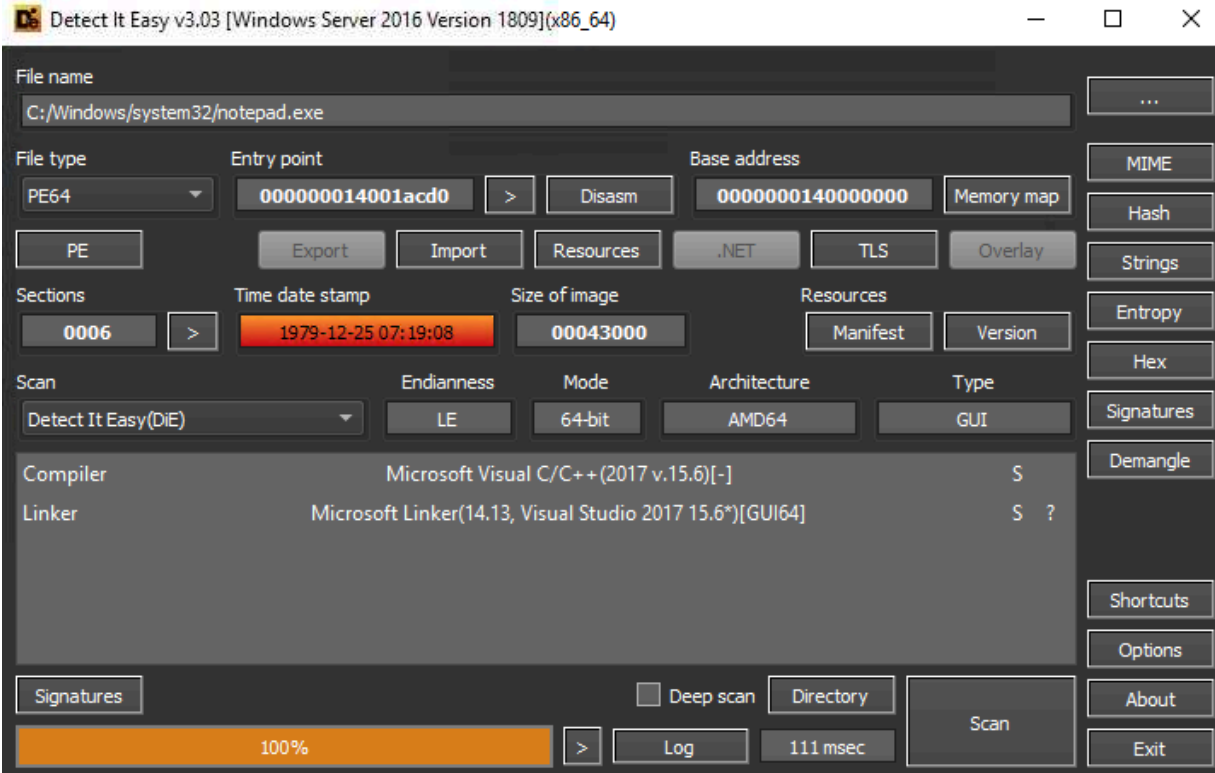
```

Section	Purpose
.text	Contains executable code and entry point
.data	Contains initialized data (strings, variables, etc.)
.rdata or .idata	Contains imports (Windows API) and DLLs.
.reloc	Contains relocation information
.rsrc	Contains application resources (images, etc.)
.debug	Contains debug information

Answer the questions below

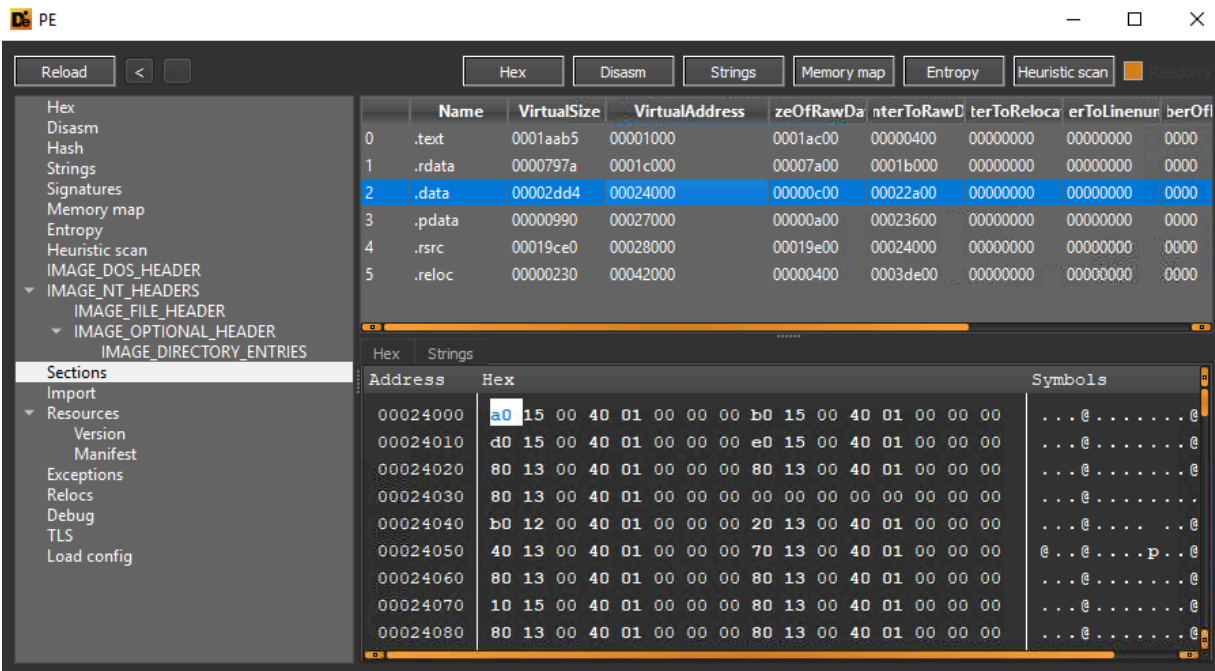
Read the above and answer the questions below. What PE component prints the message "This program cannot be run in DOS mode"? DOS Stub

Open "notepad.exe" in Detect It Easy and answer the questions below. What is the entry point reported by DiE? 000000014001acd0

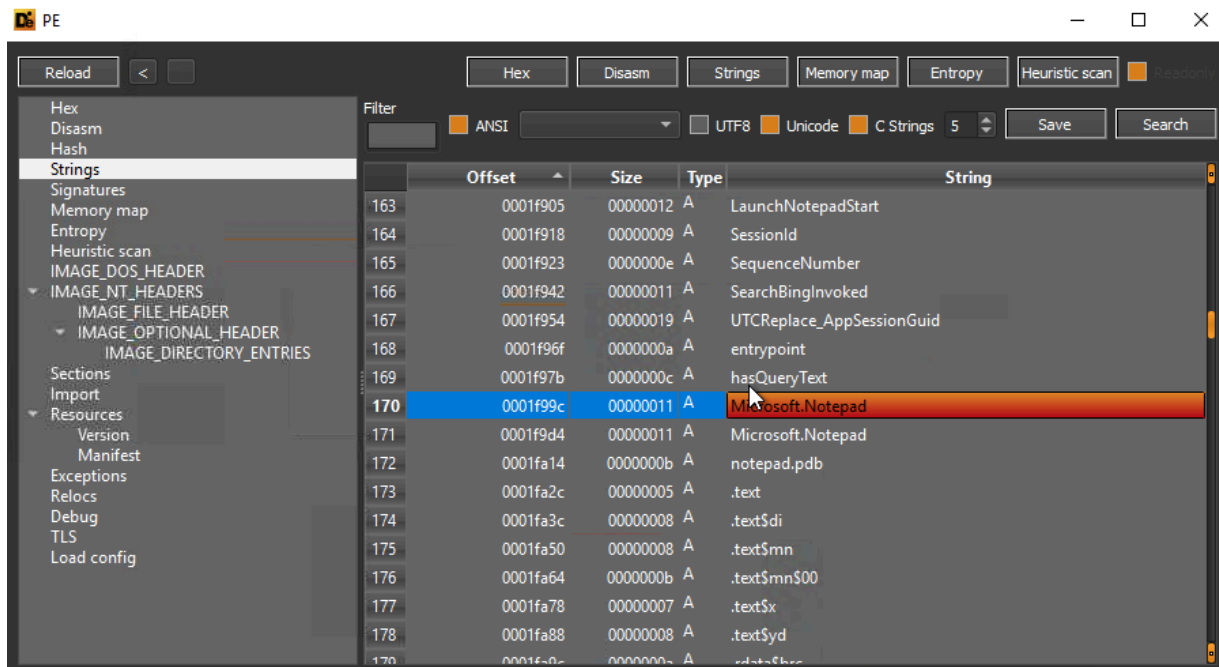


What is the value of "NumberOfSections"? 0006

What is the virtual address of ".data"? 00024000



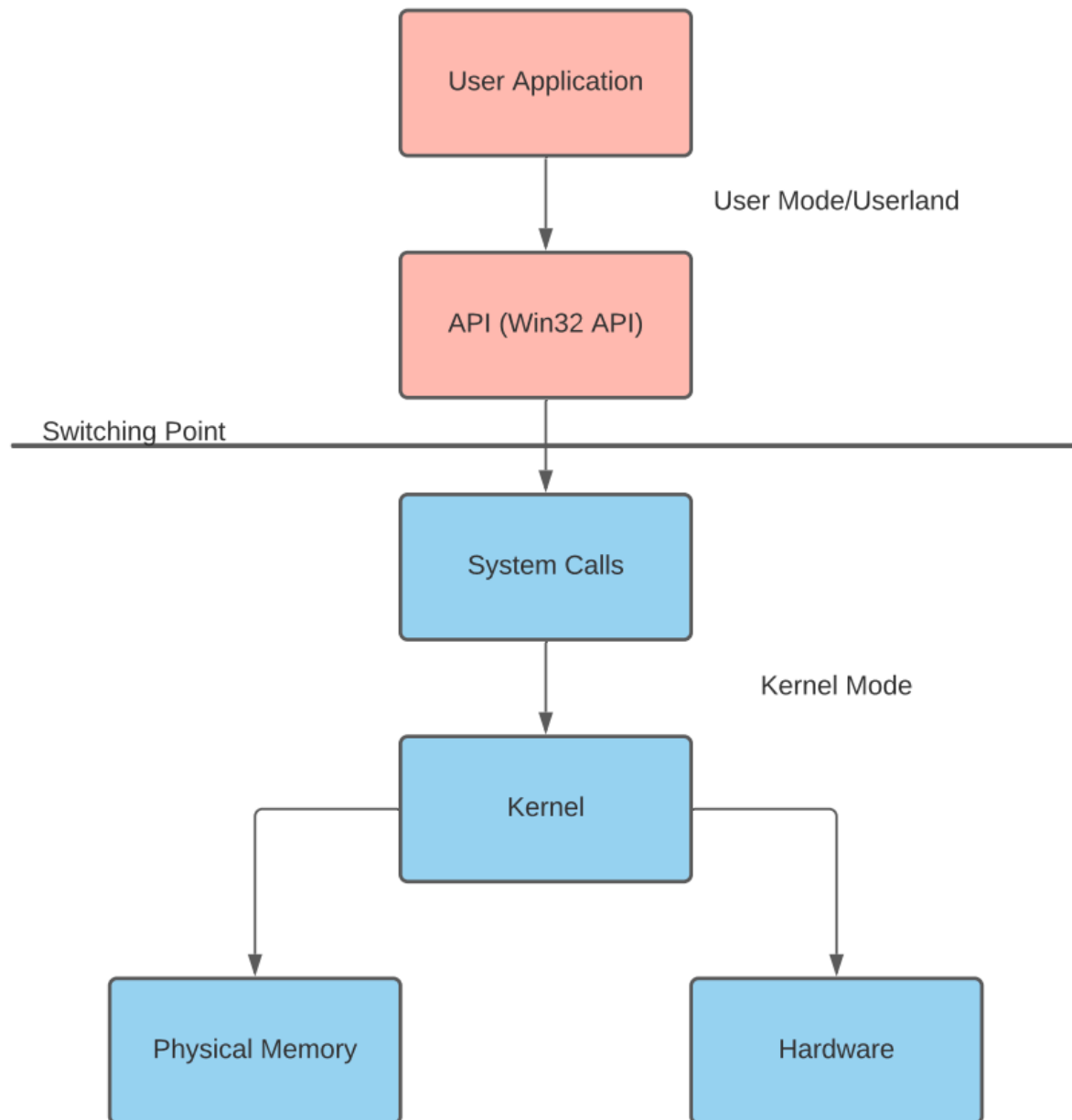
What string is located at the offset "0001f99c"? Microsoft.Notepad



Task 7 - Interacting with Windows Internals

- **Windows API:** Interacting with Windows internals is simplified through Windows API calls (primarily the Win32 API), which provide native functions to communicate with the operating system.
- **Kernel & Hardware Bridge:** The Windows kernel controls all software processes and acts as the bridge to physical hardware and memory, which standard applications cannot directly modify.
- **Processor Modes:** To maintain security and stability, the processor operates in two modes:
 - **User Mode:** Where standard applications run with restricted hardware access.
 - **Kernel Mode:** Where core OS components run with full system access.
- **The Switching Point:** System and API calls trigger transitions between User Mode and Kernel Mode when an application requires lower-level system resources.

User mode	Kernel Mode
No direct hardware access	Direct hardware access
Creates a process in a private virtual address space	Ran in a single shared virtual address space
Access to "owned memory locations"	Access to entire physical memory



- **Mode Switching:** Applications run in user mode ("userland") until a system call or API request forces a switch to kernel mode.
- **Language Runtime Layering:** Managed languages add an extra abstraction layer before reaching the Win32 API and making system calls (for example, C# code executes through the CLR first).
- **Memory Interaction Proof-of-Concept:** Demonstrating local process injection involves three core steps:
 1. **Allocate** local memory for the payload (e.g., a message box).
 2. **Write/Copy** the payload code into that allocated memory.
 3. **Execute** the payload directly from the local process memory.

Answer the questions below

Open a command prompt and execute the provided file: "inject-poc.exe" and answer the questions below. Enter the flag obtained from the executable below.
THM{1Nj3c7_4IL_7H3_7h1NG2}

Open CMD, navigate to the Process Injection directory, and run the file.

```

Microsoft Windows [Version 10.0.17763.1821]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\THM-Attacker>cd Desktop

C:\Users\THM-Attacker\Desktop>dir
Volume in drive C has no label.
Volume Serial Number is A8A4-C362

Directory of C:\Users\THM-Attacker\Desktop

11/15/2025  01:44 PM    <DIR>          .
11/15/2025  01:44 PM    <DIR>          ..
07/28/2026  07:15 AM    <DIR>          die_win64_portable_3.03
11/15/2025  01:44 PM             2,278 Google Chrome.lnk
01/16/2022  10:59 PM    <DIR>          Process Injection POC
01/15/2022  07:23 PM    <DIR>          ProcessMonitor
               1 File(s)              2,278 bytes
               5 Dir(s)  31,648,043,008 bytes free

C:\Users\THM-Attacker\Desktop>cd "Process Injection POC"

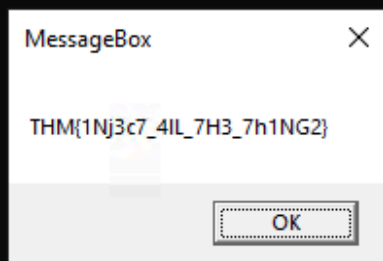
C:\Users\THM-Attacker\Desktop\Process Injection POC>dir
Volume in drive C has no label.
Volume Serial Number is A8A4-C362

Directory of C:\Users\THM-Attacker\Desktop\Process Injection POC

01/16/2022  10:59 PM    <DIR>          .
01/16/2022  10:59 PM    <DIR>          ..
01/16/2022  07:53 PM             1,559 inject-poc.cpp
01/16/2022  10:49 PM             9,728 inject-poc.exe
               2 File(s)              11,287 bytes
               2 Dir(s)  31,647,936,512 bytes free

C:\Users\THM-Attacker\Desktop\Process Injection POC>.\inject-poc.exe
shellcode length: 282

```



Task 8 - Conclusion

Windows internals are foundational components of the OS that cannot easily be changed, making them a high-value target for attackers. Security professionals on both offense (red teams) and defense (blue teams) must understand these core

mechanisms—and how they are abused—since these fundamental concepts apply broadly across Windows and Unix systems alike.